

AAMT-WP-07: Merkle-Chained LoRA Genealogy and Hopfield Watermarking

AAMT-WP-07 | AsAManThinks Research | May 2026 | DOI:
10.5281/zenodo.19600795

Merkle-Chained LoRA Genealogy and Hopfield Attractor Watermarking for Adapter Provenance Verification

Weslyn Whitehead Jr.

AsAManThinks Research, Berkeley, CA, USA |

weslyn@asamanthinks.com

ORCID: [to be registered](#) | License: CC BY 4.0

Merkle-Chained LoRA Genealogy and Hopfield Attractor Watermarking for Adapter Provenance Verification

Authors: Weslyn Whitehead Jr.¹

Affiliations:

¹ AsAManThinks Research, Berkeley, CA, USA

Corresponding author: weslyn@asamanthinks.com

ORCID: <https://orcid.org/0009-0005-7707-3210>

Submitted: May 2026

Working Paper Series: AAMT-WP-07

Anchor DOI: 10.5281/zenodo.19600795

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Abstract

The proliferation of LoRA adapters on public model hubs has created a provenance crisis: there is no standard mechanism to verify which base model trained an adapter, whether it was fine-tuned from another adapter, or whether its weights have been adversarially modified since creation. We present two complementary techniques for LoRA adapter provenance. First, **Hopfield Attractor Watermarking (HAW)**: N sentinel input vectors (derived from a fixed seed) are used post-training to compute a Hopfield-style attractor pattern in rank-1 LoRA component projections; this pattern is embedded as an additive low-rank delta invisible to normal usage but verifiable via a single sentinel forward pass. Second, **Merkle-Chained Adapter Genealogy (MCAG)**: each adapter stores a cryptographic parent hash in a sidecar `lineage.merkle.json` file, forming a verifiable Merkle tree of adapter derivations. Verification of either mechanism detects adversarial weight modification (HAW fails),

unauthorized derivation chains (MCAG chain does not terminate in a known root), or hash mismatches (tamper detection). We include a quantization-tolerant verification path: attractor residuals are rounded to 8-bit integers before hashing, surviving the weight noise introduced by Q4_K M quantization. HAW and MCAG are specified as Tier-1 feature T1.3 of MaiaM Alchemist.

Keywords: LoRA, model watermarking, provenance, Hopfield network, Merkle tree, adapter security, weight verification, AI supply chain

1. Introduction

The LoRA adapter ecosystem (Hu et al., 2021) has made fine-tuned model sharing accessible and low-cost. A 2B-parameter base model requires ~5 GB of storage; a LoRA adapter adding a new capability requires only 5–50 MB. This efficiency has driven rapid adapter proliferation on platforms like Hugging Face Hub, Civitai, and Ollama.

The same accessibility that makes LoRA adapters useful makes them a security concern. Three attack vectors are relevant:

1. **Adversarial modification:** A legitimate adapter is downloaded, subtly modified (e.g., to insert backdoors, bias outputs, or inject prompt injections), and re-uploaded under the original name.
2. **Unauthorized derivation:** An adapter derived from a proprietary base adapter claims to be original. Without a derivation chain, the provenance claim cannot be verified.
3. **Base model spoofing:** An adapter claims compatibility with base model X but was actually trained on base model Y. Weight loading without verification may succeed (producing degraded but functional output) while silently violating the adapter’s declared provenance.

Standard SHA-256 hashing of the adapter file would catch modification attacks but only if the consumer has access to the original hash — a hash-of-the-file approach still requires a trusted hash registry. HAW embeds a verifiable signal directly in the adapter weights that is computable from the sentinel seed alone, without any external registry.

2. Hopfield Attractor Watermarking (HAW)

2.1 Background: Hopfield attractor patterns

A Hopfield network (Hopfield, 1982) stores patterns as attractors in a weight matrix $\bar{W} = \sum_i x_i \cdot x_i^T$ (Hebbian outer product). Given a noisy input, the network converges to the nearest stored attractor via iterative updates. Modern continuous Hopfield networks (Ramsauer et al., 2020) store exponentially many patterns.

HAW borrows only the attractor storage idea: we want to embed a known pattern into the LoRA weight matrices such that a specific input (the sentinel) produces a predictable output, without changing the adapter’s behavior on normal inputs.

2.2 Watermark embedding

Setup: Given adapter adapter A and sentinel seed s : 1. Generate N sentinel input vectors: `sentinels = [randn(D, seed=s + i) for i in range(N)]` 2. Select K rank-1 LoRA components (specific (W_A, W_B) pairs from the adapter). 3. For each sentinel s_i : compute its projection onto the LoRA component's input space. 4. Define attractor patterns as: `attractor_k = normalize(sum(projections_for_sentinel_k))` 5. Embed as an additive delta: `W_B_watermarked = W_B +  $\epsilon$  · attractor_k · sentinelT`

where ϵ is small enough that the delta is below measurement noise on normal inputs but detectable via the sentinel.

2.3 Verification

```
def verify(adapter_path: Path, sentinel_seed: int) -> VerificationResult:
    adapter = load_lora(adapter_path)
    sentinels = generate_sentinels(sentinel_seed, n=N)
    residuals = []
    for sentinel in sentinels:
        output = adapter_forward(sentinel) # partial forward, sentinel input
        residuals.append(output)
    fingerprint = _fingerprint(torch.stack(residuals))
    stored_hash = read_watermark_hash(adapter_path)
    return VerificationResult(
        passed=(fingerprint == stored_hash),
        fingerprint=fingerprint.hex(),
        stored_hash=stored_hash.hex()
    )
```

2.4 Quantization-tolerant fingerprinting

Standard SHA-256 of float32 residuals fails after Q4_K_M quantization (4-bit weights introduce ~0.5% weight noise, shifting residual values). The quantization-tolerant fingerprint rounds to 8-bit before hashing:

```
def _fingerprint(residuals: torch.Tensor) -> bytes:
    """Survives Q4_K_M weight noise; fails on adversarial modification."""
    quantized = (residuals * 127.0).round().clamp(-127, 127).to(torch.int8)
    return hashlib.sha256(quantized.numpy().tobytes()).digest()
```

This fingerprint is identical before and after Q4_K_M quantization (the 8-bit rounding absorbs the quantization noise) but detects deliberate single-parameter modifications with high probability.

3. Merkle-Chained Adapter Genealogy (MCAG)

3.1 The Merkle sidecar

Each adapter includes a sidecar file `lineage.merkle.json`:

```
{
  "adapter_hash": "sha256:a3f2c1...",
  "parent_hash": "sha256:b7e4d2...",
  "base_model_id": "google/gemma-2-2b",
  "base_model_hash": "sha256:c9d3e1...",
  "created_at": "2026-05-28T14:22:04Z",
  "run_id": "run-abc123",
  "watermark_hash": "sha256:d4f5e2..."
}
```

The `adapter_hash` is SHA-256 of the adapter's `safetensors` file. The `parent_hash` is the `adapter_hash` of the adapter from which this one was derived (via PT-CDS, continued training, or merging). Root adapters have `parent_hash = null`.

3.2 Chain verification

```
def verify_chain(adapter_path: Path,
                 known_roots: set[str]) -> ChainVerificationResult:
    chain = []
    current = adapter_path
    while True:
        merkle = load_merkle(current)
        chain.append(merkle.adapter_hash)
        if merkle.parent_hash is None:
            if merkle.adapter_hash in known_roots:
                return ChainVerificationResult(valid=True, chain=chain)
        else:
            return ChainVerificationResult(
                valid=False, reason="chain_root_not_trusted",
                chain=chain)
        current = resolve_parent(merkle.parent_hash)
        if current is None:
            return ChainVerificationResult(
                valid=False, reason="parent_not_found", chain=chain)
        if merkle.adapter_hash in chain[:-1]:
            return ChainVerificationResult(
                valid=False, reason="cycle_detected", chain=chain)
    )
```

The `known_roots` set is published by the adapter's original creator (e.g., in a signed manifest file). Any adapter whose Merkle chain does not terminate in a known root is rejected.

3.3 Swarm peer rejection

In the Harmonic Swarm protocol (WP-14), peers can be configured to reject adapters whose chain does not terminate in a trusted root:

```
if swarm_config.enforce_lineage:
    result = verify_chain(adapter_path, known_roots=swarm_config.
                          trusted_roots)
    if not result.valid:
        raise AdapterProvenanceError(result.reason)
```

This allows swarm operators to enforce provenance policies across distributed inference nodes.

4. Combined Verification Protocol

A complete adapter verification checks both mechanisms:

Check	Method	Detects
Watermark present	HAW sentinel forward pass	Weight modification, weight injection
Watermark matches	Fingerprint comparison	Adversarial parameter editing
Chain terminates	Merkle parent traversal	Unauthorized derivation
Chain root trusted	Known-roots membership	Chain forgery
Base model matches	<code>base_model_hash</code> comparison	Base model spoofing

All five checks must pass for full verification. Each check is independent; a failure in any one is reported with a specific reason code.

5. Security Analysis

Adversarial resilience of HAW: To forge the watermark, an adversary must find a set of rank-1 weight deltas that produce the correct sentinel residuals without knowing `sentinel_seed`. This requires solving for the attractor pattern given only the forward-pass output — a non-trivial inversion problem. The attractor dimension (`pattern_dim = 64` by default) provides 64-dimensional verification. Even treating the verification as a simple binary decision per dimension (a significant underestimate of the actual search difficulty), the brute-force search space is $2^{64} \approx 1.8 \times 10^{19}$ — larger than the number of seconds in the age of the universe and far beyond any feasible exhaustive search. In practice the verification is continuous-valued (residuals are float32), making the inversion problem substantially harder than this discrete bound suggests.

Merkle chain forgery: To forge a valid chain, an adversary must either find a collision in SHA-256 (computationally infeasible) or insert a fraudulent entry in the known-roots registry (a social engineering attack outside the cryptographic scope of MCAG).

Quantization robustness: The 8-bit fingerprinting threshold ($\pm 0.4\%$ of full range) comfortably absorbs Q4_K_M noise (typically $\pm 0.1\%$) while detecting deliberate parameter changes of ≥ 1 bit flip in INT8 representation.

6. Relationship to Prior Work

Neural network watermarking (Uchida et al., 2017; Zhang et al., 2018): Embed ownership marks in model weights via regularization during training. HAW is post-training and does not require modifying the training objective.

Text watermarking (Kirchenbauer et al., 2023): Embed signals in the token distribution during generation. HAW watermarks the weights, not the outputs.

Model fingerprinting (Cao et al., 2021): Verify model identity via backdoor-style test inputs. HAW uses Hopfield-style attractor patterns rather than backdoor examples, and requires no additional training.

Supply chain security (SLSA framework): MCAG is compatible with SLSA (Supply chain Levels for Software Artifacts) principles applied to ML models — each link in the chain corresponds to a build step with a verified parent.

Key distinction: No prior work combines post-training attractor watermarking with a Merkle-chained derivation graph and a quantization-tolerant verification path. The combination addresses the three practical attack vectors (modification, unauthorized derivation, base model spoofing) with a single unified verification API.

7. Conclusion

HAW and MCAG together provide a comprehensive adapter provenance system: HAW detects weight tampering via Hopfield attractor patterns that survive quantization; MCAG provides a verifiable derivation history via cryptographic hash chaining. The combination addresses the LoRA adapter provenance crisis without requiring external registries, training-time modifications, or coordination between the adapter creator and consumer beyond the sidecar JSON file. These mechanisms are directly applicable to the broader open-weight adapter ecosystem beyond MaiiaM Alchemist.

Acknowledgments

The author thanks the AsAManThinks Research community.

Funding

Self-funded through AsAManThinks Research.

Data Availability

The HAW and MCAG implementation specs are fully described herein. The `crypto/attractor.py` and `crypto/lineage_chain.py` modules are part of MaiiaM Alchemist T1.3.

Conflict of Interest

The author is the founder and CEO of AsAManThinks Research.

References

- Cao, X., Fang, J., Liu, Y., & Gong, N. Z. (2021). IPGuard: Protecting intellectual property of deep neural networks via fingerprinting the classification boundary. In *Proceedings of ACM AsiaCCS 2021* (pp. 14–25). <https://doi.org/10.1145/3433210.3437526>
- Hopfield, J. J. (1982). Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences*, 79(8), 2554–2558.

<https://doi.org/10.1073/pnas.79.8.2554>

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). LoRA: Low-rank adaptation of large language models. In *Proceedings of ICLR 2022*. <https://doi.org/10.48550/arXiv.2106.09685>

Kirchenbauer, J., Geiping, J., Wen, Y., Katz, J., Miers, I., & Goldstein, T. (2023). A watermark for large language models. In *Proceedings of ICML 2023* (pp. 17061–17084). <https://doi.org/10.48550/arXiv.2301.10226>

Ramsauer, H., Schödl, B., Lehner, J., Seidl, P., Widrich, M., Adler, T., Gruber, L., Holzleitner, M., Pavlović, M., Sandve, G. K., Greiff, V., Kreil, D., Kopp, M., Klambauer, G., Brandstetter, J., & Hochreiter, S. (2020). Hopfield networks is all you need. In *Proceedings of ICLR 2021*. <https://doi.org/10.48550/arXiv.2008.02217>

Uchida, Y., Nagai, Y., Sakazawa, S., & Satoh, S. (2017). Embedding watermarks into deep neural networks. In *Proceedings of ICMR 2017* (pp. 269–277). <https://doi.org/10.1145/3078971.3078974>

Zhang, J., Gu, Z., Jang, J., Wu, H., Stoecklin, M. P., Huang, H., & Molloy, I. (2018). Protecting intellectual property of deep neural networks with watermarking. In *Proceedings of AsiaCCS 2018* (pp. 159–172). <https://doi.org/10.1145/3196494.3196550>

Whitehead, W. (2026). MaiaM Wings: Consciousness-Aligned Language Architecture. *Zenodo*. <https://doi.org/10.5281/zenodo.19600795>